Inference

Lecture 4: what it costs to run a language model

Aarne Talman

10 November 2026

LDA-T318 | LLMs and Generative AI for NLP | University of Helsinki

Course overview

Wk	Dates	Tuesday	Thursday	Teacher
1	27 / 29 Oct	Language models	Pretraining	Aarne
2	3 / 5 Nov	Post-training	Lab 2: tune vs prompt	Aarne
3	**10 / 12 Nov**	**Inference**	**Lab 3: serve**	**Aarne**
4	17 / 19 Nov	Evaluation	Lab 4: evaluate	Jussi
5	24 / 26 Nov	RAG	Lab 5: RAG	Dmitry
6	1 / 3 Dec	Agents	Lab 6: agent	Dmitry
7	8 / 10 Dec	Course recap	Special topics (TBC)	All

Reusing a task and model across labs is recommended, not required.

There are no intermediate submissions; the final report and code repository determine the grade.

Lecture overview

1. Decoding *10 min*
2. Prefill vs decode *10 min*
3. The KV cache *15 min*
4. Continuous batching *10 min*
5. Quantisation for inference *10 min*
6. Speculative and constrained decoding *10 min*
7. Serving systems and cost *15 min*
8. Lab 3: measuring a server fairly *10 min*

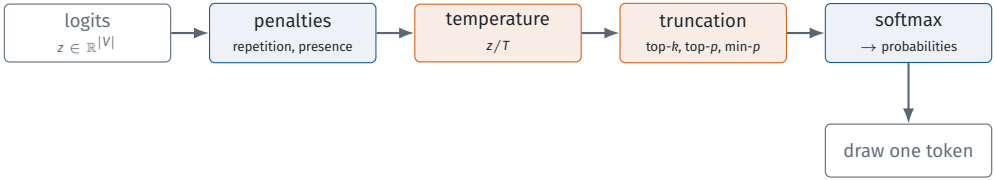
Take away

By the end of today you should be able to explain where inference time and memory go, estimate a model's serving capacity, and design a fair speed–quality comparison.

Decoding

From logits to a token

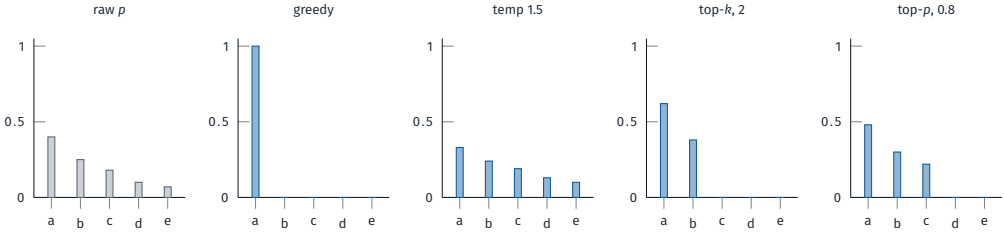
Every generated token goes through the same short pipeline. All of it happens *after* the forward pass, on a vector of $|V| \approx 150{,}000$ numbers.



Two things worth noticing now:

- it is **cheap**: a few operations over one vector, next to billions of FLOPs in the forward pass;
- it is **not part of the model**. The same weights behave very differently under different settings, and those settings live in `generation_config.json`, in your client call, or in your server's defaults.

Temperature and truncation change one distribution



greedy	take the most likely token.
temperature	values below 1 sharpen the distribution; values above 1 flatten it.
top-k	keep a fixed number of likely tokens, then renormalise.
top-p	keep the smallest set whose probabilities add up to p; the set adapts to confidence.

Which sampler for which task

Task	Sensible default	Why
Classification, extraction, exact-match QA	temperature 0 (greedy)	there is one right answer, so variance only costs you
Code, maths, anything verified	temperature 0 for one shot; about 0.8 if you sample several and vote	diversity only pays when you have a verifier to pick the winner
Open generation, dialogue, summarisation	temperature 0.7, with top-p 0.9 or min-p 0.05	greedy output is repetitive and flat, and a human is judging

Sample widely only when a human or an automatic checker can reject poor outputs. Otherwise sampling adds variance without a way to select the better result.

Source: Raschka, *Build Your Own Reasoning Model*, Oct 2026, on best-of-n and self-consistency.

Decoding parameters are part of your experimental setup

Three places a sampling parameter can come from, in increasing order of surprise:

1. your API call
2. the server's defaults
3. the model's own `generation_config.json`, shipped in the repository and silently applied by `transformers`

They disagree. `transformers` honours the model's config; a server may or may not; OpenAI-compatible clients have their own defaults for `temperature` and `top_p`.

The rule for weeks 4–6

Set decoding parameters **explicitly**, in one place, and record them next to every number you report. A speed-versus-quality table is meaningless if the two rows were generated at different temperatures, and that is the easiest mistake to make by accident in Thursday's lab.

Also fix `max_tokens` and the random seed, and report the *distribution* of your measurements, not only the mean.

Reasoning on or off: what thinking tokens cost

A reasoning model pays for its accuracy in **output tokens**, which are the expensive kind, because each one costs a full forward pass.

- Effort level correlates with token usage, and token usage correlates with accuracy [Raschka, Jul 2026]
- Kimi's *Toggle* training cuts generated tokens by **25–30%** with little benchmark change, and the saving transfers from maths and code RL to GPQA and MMLU-Pro
- GPT-5.6 *Luna* uses **80% more tokens** than *Sol* at comparable scores: a smaller model buys the same result with more thinking

Worked, for a serving budget

Answer without thinking: 150 output tokens.
Answer with thinking: 1,800 output tokens.

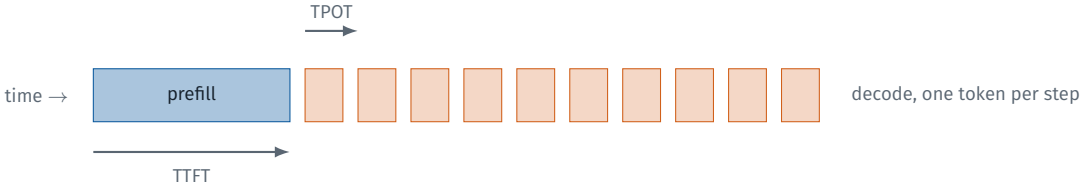
At a per-user TPOT of 5 ms that is **0.75 s** versus **9 s** of GPU-seconds for the *same question*, a 12× difference in both latency and cost.

And the KV cache grows with every one of those tokens, so a reasoning request also occupies more memory for longer.

Sources: Raschka, *Controlling Reasoning Effort in LLMs*, Jul 2026; Kimi K2.5 / Toggle report; Artificial Analysis Intelligence Index v4.3, as reported in Raschka, Jul 2026.

Prefill vs decode

One request, two different computations



	Prefill	Decode
What	the whole prompt at once	one new token per step
Shape	matrix × matrix	matrix × *vector*
Reads	every weight once, for 2,000 tokens	every weight once, for 1 token
Limited by	**how fast the chip multiplies**	**how fast memory feeds it**
Felt as	**TTFT** — wait for the first token	**TPOT** — gap between tokens

Fetching a number costs far more than multiplying it

A GPU does two things, and they are nothing like equally expensive:

- **fetching** a weight out of the card's memory into the arithmetic units — **slow**;
- **multiplying** with it once it has arrived — roughly a hundred times cheaper.

So the number that decides your speed is: *how much arithmetic do you get out of each byte you went to the trouble of fetching?* That ratio is called **arithmetic intensity**, measured in FLOPs per byte read.

Decode: one token at a time

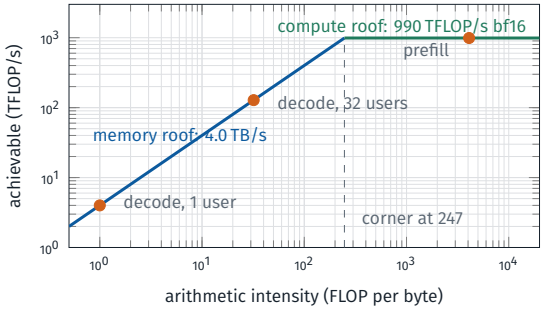
Read all 8 GB of weights and use each weight in one multiply-add per token. Batch 32 users and each fetched weight serves 32 tokens. Intensity ≈ the number of tokens in the step.

Prefill: the whole prompt at once

A 2,000-token prompt goes through in one pass. The same 8 GB is read once and reused two thousand times over. Intensity ≈ 2000.

For ordinary decode, batching is the main way to reuse fetched weights.

The roofline



A **roofline** is a ceiling chart: whatever your code does, it cannot go above either line. The sloped part is the limit set by how fast memory feeds the chip. The flat part is where the multipliers themselves become the limit. This card can do about 990 trillion floating-point operations a second but fetch only 4 terabytes a second, so it is not even nominally busy until you hand it about **250 FLOPs per byte**, which works out at **roughly 250 tokens in every decode step.**

Source: Roofline model: Williams, Waterman & Patterson, CACM 2009. Hardware figures: NVIDIA GH200 (H100-class, 96 GiB HBM3) product specifications.

A BANDWIDTH ESTIMATE FOR DECODE SPEED

$$\text{tokens per second} \lesssim \frac{\text{memory bandwidth}}{\text{bytes of weights}}$$

In words: how many times per second can the card read the whole model?

Your project model, 4B in bf16, on one GH200:

- 4 billion weights, 2 bytes each: **8 GB** to read for every single token
- the card moves **4.0 TB** of memory per second
- 8 GB goes into 4.0 TB about **500 times**, so 500 tokens a second, i.e. 2 ms each

Include the KV cache At 8k context, this example also reads about 1.2 GB of cached keys and values each step. The ceiling becomes

$$4.0\,\mathrm{TB/s} \div 9.2\,\mathrm{GB} \approx \textbf{430 tokens/s}.$$

That is the *ceiling* for a single stream, however good the kernels are.

Source: NVIDIA GH200 Grace Hopper Superchip datasheet (HBM3 and HBM3e variants); CSC, `docs.csc.fi/computing/systems-roihu`.

From the ceiling to a measured speed

	tokens/s	why lower
Paper ceiling, weights only	500	—
Ceiling including an 8k KV cache	≈ 430	you re-read the cache every step
Realistic achieved bandwidth ($\approx 70\%$)	≈ 300	imperfect access patterns
What a server actually reports, $B = 1$	**150–250**	scheduler, Python, HTTP, sampling

Measure single-stream TPOT, invert it, and compare the result with the estimate. A result within a factor of two is useful agreement for this simplified model. Then repeat at concurrency 8 and 32: aggregate throughput should rise while each request receives a smaller share of the server.

TTFT, TPOT, AND WHO FEELS WHICH

$$\text{total wait} = \underbrace{\text{TTFT}}_{\text{reading the prompt}} + \underbrace{\text{TPOT} \times (\text{tokens after the first})}_{\text{generating the answer}}$$

In words: you wait once for the prompt, then once per token produced.

Worked, 4B model, 2,000-token prompt, 500 output tokens:

- prefill: pushing 2,000 tokens through a 4B model is about 16 trillion floating-point operations
- the card really achieves about 400 trillion a second, so: **40 ms**
- decode: 500 tokens at 5 ms each is **2.5 s**

Prefill is **1.6%** of the wall clock. Two thousand prompt tokens cost about as much time as **eight** generated ones.

Who feels what

TTFT — chat users, agents doing many short calls, anything with a spinner.

TPOT — anyone reading a streamed response. Anything under about 30 ms per token feels instant, and by 100 ms it starts to feel broken.

Neither — batch jobs. For offline scoring only *throughput* matters, and you should batch as hard as the memory allows.

Which technique helps which phase

Technique	helps prefill	helps decode
Bigger batches	no (already busy)	**yes, hugely**
INT4 weight-only quantisation	**can hurt**	**yes**
FP8 weights + activations	**yes**	**yes**
KV-cache quantisation	slightly	**yes**
Prefix caching	**yes, enormously**	no
Speculative decoding	no	**yes**

Take away

Work out which half of the request you are paying for before you optimise.

The KV cache

Week 1's formula, read as a budget

$$\text{KV bytes} = \text{requests} \times \text{tokens} \times \text{layers} \times \text{KV heads} \times \underbrace{\text{numbers per head}}_{128} \times \underbrace{2}_{\text{key and value}} \times \underbrace{2}_{\text{bytes each, bf16}}$$

In words: every layer stores two small vectors for every token of every request, and you pay for all of them at once. Your project model has 36 layers and 8 KV heads, so

$$36 \times 8 \times 128 \times 2 \times 2 \;\approx\; 147\,\mathrm{kB} \text{ per token}$$

In week 1 this was an architecture slide

It explained why GQA, MLA, SWA and cross-layer sharing exist.

Today it is a capacity planning slide

It tells you how many people you can serve, and it is the number your server runs out of first.

And this is not only a storage cost. Every decode step *reads the whole cache back*, so it eats bandwidth too.

The same model at 8k and at 128k

Context	KV / request	× weights (8 GB)	of 96 GiB
1 request @ 8k	1.2 GB	0.15×	1%
1 request @ 32k	4.8 GB	0.6×	5%
1 request @ 128k	19.3 GB	2.4×	19%

One user at 128k context occupies **more than twice the memory of the entire model**.

The relationship is linear: double the context and you double the cache, which halves the number of users you can hold. There is no economy of scale in that formula.

Units, stated once

96 GiB = 103 GB.
A server typically reserves ≈ 90% of the card, and the weights take 8 GB, so

$$0.9 \times 103 - 8 \approx \mathbf{85}\,\mathrm{GB}$$

is what is actually available for KV. We use 85 GB from here on.

Multiply by concurrency until it breaks

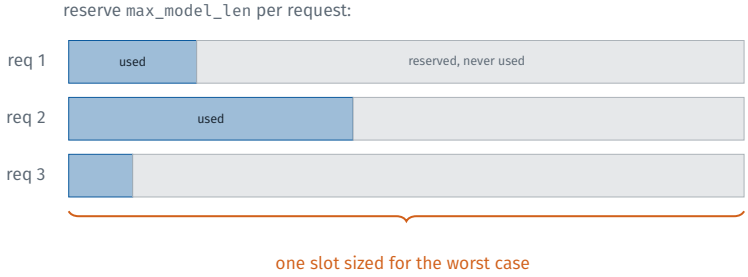
Concurrent	KV @ 8k	+ 8 GB weights	
1	1.2 GB	9.2 GB	trivially fine
8	9.6 GB	17.6 GB	fine
32	38.6 GB	46.6 GB	fine — [week 1's number]
64	77 GB	85 GB	at the limit
70	84 GB	92 GB	over budget

At **128k** the same arithmetic gives **4** concurrent requests: sixteen times the context buys you sixteen times fewer users. The card still has arithmetic to spare and nowhere left to put anyone.

The conclusion of this section

In this example, the KV cache sets the concurrency limit. The 4B model's weights take 8 GB, while its cache can grow to many times that at long contexts and high concurrency.

Where the memory actually goes under naive allocation



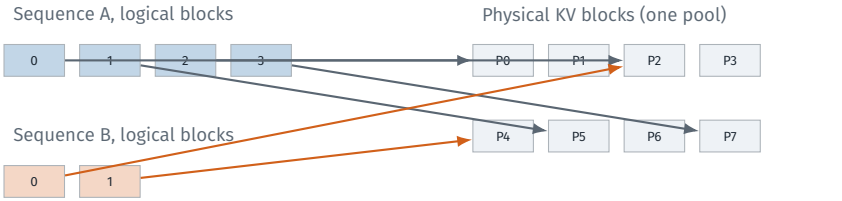
Three kinds of waste, all of them avoidable:

- **internal** — you reserved 8k and the answer was 200 tokens
- **reservation** — memory held for tokens that may never be generated
- **external fragmentation** — free memory exists but not in one contiguous run of the right size

Measured waste in pre-paged serving systems: **60–80%** of the KV region.

Source: Kwon et al., *Efficient Memory Management for LLM Serving with PagedAttention*, SOSP 2023.

PagedAttention



- KV is stored in fixed-size **blocks** (16 tokens is typical), allocated on demand
- a per-sequence **block table** maps logical to physical blocks — they need not be contiguous
- waste is now at most one partly-filled block per sequence: **under 4%**
- identical prefixes can **share** a physical block; a block is copied only when one sequence writes to it

Source: Kwon et al., SOSP 2023 — the paper vLLM is built on.

Prefix caching: paying for the system prompt once

If a prompt **begins with** tokens whose KV blocks are still cached, the server reuses them and skips that part of prefill.

Systems such as SGLang and vLLM can find and reuse a shared prefix across many requests.

This does not intentionally approximate the model, but it consumes cache memory. The gain depends on whether the prefix remains resident.

Worked: a 5-shot evaluation run

Shared prefix: system prompt + 5 examples ≈ 1,200 tokens.
Per-item suffix: ≈ 80 tokens.

Without prefix caching you prefill 1,280 tokens per item. With it, **80**.

Over a 1,000-item test set that is up to 16× **less prefill work** while that prefix remains cached.

The same idea helps multi-turn chat, agent loops and fixed RAG instructions.

Source: Zheng et al., *SGLang: Efficient Execution of Structured Language Model Programs*, 2024 (RadixAttention); vLLM automatic prefix caching.

Capacity, summarised, with one caveat

Context	KV / request	max concurrent, bf16 KV	with FP8 KV
8k	1.2 GB	70	140
32k	4.8 GB	17	35
128k	19.3 GB	4	8

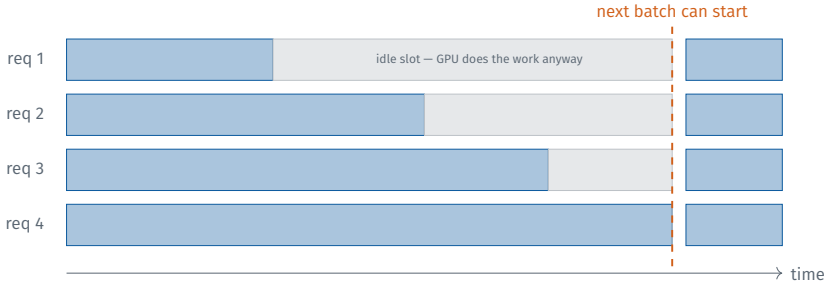
One GH200, 4B model in bf16, 85 GB available for KV.

A practitioner's caveat, worth repeating all term

A model with a theoretically cheaper attention mechanism is not automatically cheaper to *serve*. Inference stacks have years of optimisation behind plain GQA and far less behind hybrid and linear attention. In late 2026 a "slower" classic GQA model can still serve faster in practice than a hybrid one. Until the kernels catch up, **the better-supported method usually wins**.

Continuous batching

Static batching wastes most of the GPU

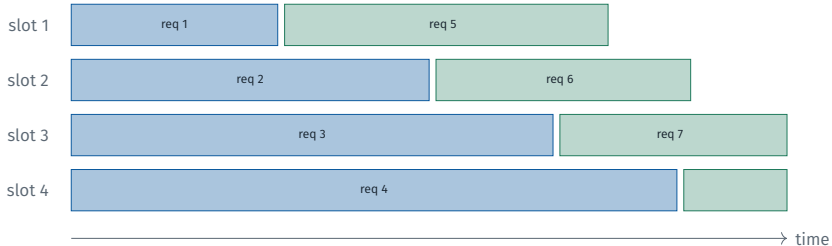


- The batch is fixed when it starts and released when the *slowest* member finishes.
- A finished sequence still occupies a slot; the kernel computes padding for it.
- Request 5 waits behind request 4's last token even though three slots are free.

Output lengths vary by an order of magnitude in real traffic. A one-line answer and a 4,000-token reasoning trace end up in the same batch, so this waste is the normal case.

Source: Yu et al., *Orca: A Distributed Serving System for Transformer-Based Generative Models*, OSDI 2022.

Continuous (iteration-level) batching



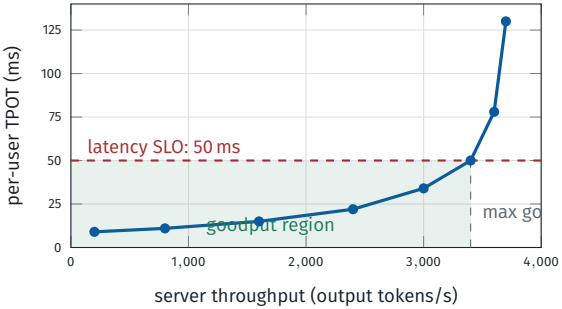
The scheduler rebuilds the batch **every decoding iteration**, not every request:

- a sequence that emits its stop token leaves immediately and frees its KV blocks
- a waiting request joins at the very next step, prefill included
- padding, idle slots and head-of-line blocking all disappear

Reported throughput gains over static batching at the same latency: roughly 2–4×, before PagedAttention is even considered.

Source: Yu et al., OSDI 2022; Kwon et al., SOSP 2023.

Throughput and latency pull in opposite directions



Raise concurrency and aggregate throughput climbs, until the knee, where each added request slows everyone without adding tokens. **Goodput** is throughput that still meets the latency target, and past the knee extra load produces only violations.

Source: Shape after published vLLM and DistServe throughput–latency sweeps; values illustrative.

Chunked prefill

The problem. A 32k-token prompt arrives. Its prefill is one large, compute-bound forward pass. For its duration, every other user's decoding stops.

One 32k prefill at ≈ 400 TFLOP/s is roughly **650 ms**, and that shows up as a 650 ms stall in somebody else's token stream, for a request they have nothing to do with.

The fix. Split the prefill into chunks (for example 512 tokens) and schedule each chunk *alongside* the decode steps of the running batch.

Why it also helps throughput

A decode-only step spends its time waiting on memory and leaves the arithmetic units idle. A prefill chunk is the opposite kind of work: plenty of arithmetic, hardly any new weights to fetch. **Mixing them in one batch uses both resources at once.**

The cost: TTFT for the long prompt gets slightly worse, because its prefill is now interleaved rather than monolithic. You are trading one user's TTFT for everyone else's TPOT, deliberately.

Source: Agrawal et al., *Taming Throughput–Latency Tradeoff in LLM Inference with Sarathi-Serve*, OSDI 2024; implemented in vLLM and SGLang.

The flags that matter

```
# annotated; in the lab these are all on one line
vllm serve Qwen/Qwen3.5-4B-Instruct
  --max-model-len 8192           # caps KV per request: 1.2 GB here
  --max-num-seqs 64              # concurrency ceiling (slots)
  --max-num-batched-tokens 8192  # tokens per scheduler step = prefill chunk
  --gpu-memory-utilization 0.90  # fraction of the card vLLM may claim
  --enable-prefix-caching        # free, turn it on
  --enable-lora --lora-modules proj=./lora-week2
  --port 8000
```

`max_model_len`	the biggest lever on concurrency; do not leave it at the model's 128k default if your task uses 2k
`max_num_seqs`	raising it past what KV memory allows just moves the queue inside the server
`max_num_batched_tokens`	bigger ⇒ better throughput, worse tail TPOT
`gpu_memory_utilization`	on a shared GH200, lower it or your neighbour's job dies

Goodput: throughput that met the deadline

Throughput — tokens per second, summed over all users. Easy to maximise: raise concurrency until memory runs out.

Goodput — tokens per second *from requests that met the latency target*. Everything else is counted as zero.

Under a 50 ms TPOT target, a server at 3,700 tokens/s and 130 ms has **a throughput of 3,700 and a goodput of 0**.

How to report honestly

- warm up before timing
- fix the output length
- separate TTFT from TPOT
- sweep concurrency: 1 / 8 / 32
- report p50 **and** p95, not the mean

Take away

Continuous batching fills GPU time that was being wasted. It gives you no extra memory, so the ceiling is still the KV cache, and someone still has to choose where on the curve you want to sit.

Quantisation for inference

Three independent places to use fewer bits

What	Size, 4B model	What lowering it buys
Weights	8 GB in bf16	fewer bytes read per decode step ⇒ directly faster decode
Activations	small, transient	lets you use the FP8 tensor cores ⇒ faster *prefill*
KV cache	1.2 GB per 8k request	more concurrent users, and less to re-read every step

These are **orthogonal choices**. “The model is quantised” is as uninformative as “the model uses sliding-window attention” was in week 1. Always ask *which of the three* uses fewer bits, and in what format.

FP8 on a GH200

What it is

- weights and activations use 8-bit floating-point numbers;
- each tensor or block also stores a scale, because FP8 represents a much narrower range than bf16;
- smaller blocks use more scales but preserve outliers better.

Hopper tensor cores multiply FP8 directly, so this is a native hardware format rather than compressed storage that must first be unpacked.

Why it is used

- weights halve: 8 GB $\rightarrow$ 4 GB;
- less weight traffic speeds up decode;
- faster FP8 tensor cores can also speed up prefill.

How to evaluate it Re-run the same held-out scorer and report quality beside latency and throughput. Published average losses do not guarantee that your language, task or long-context workload is unaffected.

Source: NVIDIA Hopper FP8 documentation; vLLM FP8 quantisation guide.

INT4 WEIGHT-ONLY: AWQ AND GPTQ

Store each weight in 4 bits, **unpack it back to 16-bit on the GPU just before use**, and do the arithmetic in bf16. Only the storage shrinks; the activations stay full precision.

GPTQ — goes layer by layer on the finished model, choosing how to round each weight so the layer's *output* changes as little as possible on a sample of real data.

AWQ — observes that a small fraction of weight channels matter far more, identifies them from *activation* magnitudes, and rescales so those channels survive quantisation.

Sources: Frantar et al., *GPTQ*, ICLR 2023; Lin et al., *AWQ*, MLSys 2024.

Why weight-only works at all

It helps for exactly the reason section 2 gave: decode is waiting on memory. Shrinking the weights from 8 GB to 2 GB means a quarter of the bytes per token, so the head-estimate ceiling goes from 500 to **2,000 tokens/s**. When decode is strongly bandwidth-bound, much of the unpacking cost can be hidden by arithmetic units that would otherwise be idle.

Both need **calibration data** — a few hundred sequences. Use data that looks like your task. Calibrating a Finnish NER model on English web text is a real and common own-goal.

Why INT4 can slow down prefill

Weight-only INT4 can speed up decode and **slow down prefill**. Decode benefits from reading fewer weight bytes. Prefill is compute-bound, so unpacking 4-bit weights to bf16 adds work to the phase that is already short of arithmetic capacity. FP8 is multiplied directly by Hopper tensor cores and can therefore help both phases.

	bytes/token	decode	prefill
bf16 (W16A16)	8 GB	baseline	baseline
FP8 (W8A8)	4 GB	$\approx 1.8\times$	$\approx 1.5\times$
INT4 (W4A16)	2 GB	$\approx 3\times$	0.8–$1.0\times$

Indicative only: the prefill number depends heavily on the kernel and the prompt length. Read the *sign* rather than the magnitude.

Source: Behaviour reported across vLLM, TensorRT-LLM and llama.cpp kernel benchmarks; see also Lin et al., AWQ, MLSys 2024.

Quantising the KV cache

The third, independent knob. Store the cached keys and values in 8-bit FP8 instead of 16-bit bf16, and every number in the cache halves: 147 kB per token becomes **73.5 kB**.

- 8k context: 1.2 GB $\rightarrow$ 0.6 GB per request
- concurrent users at 8k: 70 $\rightarrow$ **140**
- concurrent users at 128k: 4 $\rightarrow$ **8**
- and half the bytes re-read every decode step

INT4 caches exist too, but need a finer-grained set of scale factors to hold up.

Where the damage shows up

Long-context behaviour can be **more sensitive** to KV quantisation: early tokens' quantised keys may be attended to thousands of times. So the headline benchmark at 2k context looks untouched, and the 128k-context retrieval task quietly degrades. **Short-context evaluation cannot detect this failure.**

Position encoding makes the keys harder to quantise than the values: rotating them spreads their values out.

Source: Hooper et al., *KVQuant*, NeurIPS 2024; vLLM FP8 KV cache documentation.

The quality cost, and where it shows up

For FP8 and good INT4 the published quality loss is usually **under a point** on MMLU-style benchmarks. That average hides where the damage actually lands:

- **long-tail inputs**: rare formats, unusual prompts
- **non-English text**, especially morphologically rich languages
- **long reasoning chains**, where one bad token derails the rest
- **long contexts**, if you quantised the KV cache

Which is why Thursday makes you score it

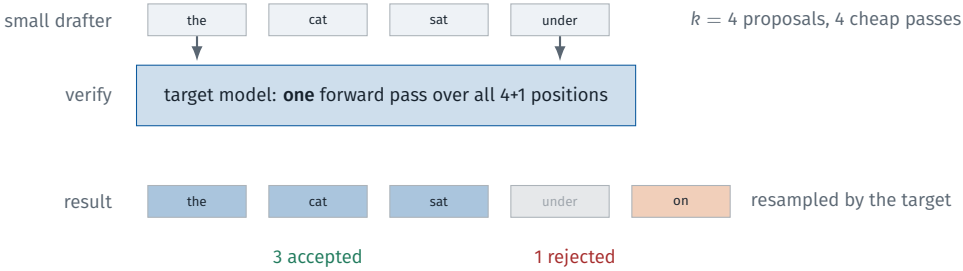
Serve the model in FP8, run the same held-out scorer used in week 2, and put the score in the same table as the tokens per second. **Always report the quality number next to the speed number.**

Take away

FP8 is the safe default on a GH200 and INT4 is a bet on decode. Either way, the only quality number worth anything is the one you measured on your own task.

Speculative and constrained decoding

Speculative decoding



Four target-model tokens' worth of progress from **one** target forward pass. The target pass costs almost the same as a single-token pass, because verifying 5 positions is still a tiny batch on a bandwidth-bound GPU, and the weights are read once either way.

Sources: Leviathan et al., *Fast Inference from Transformers via Speculative Decoding*, ICML 2023; Chen et al., *Accelerating LLM Decoding with Speculative Sampling*, 2023.

Why the output distribution is unchanged

The drafter proposes a token. The big model already computed how likely *it* thinks that token was. Compare the two numbers:

$$\text{accept with probability } \min\left(1,\ \frac{\text{big model's probability of the token}}{\text{drafter's probability of the token}}\right)$$

In words: keep the guess outright if the big model likes it at least as much as the drafter did; otherwise keep it only sometimes. If it is rejected, draw the replacement from the **residual** — the part of the big model's distribution the drafter had been under-weighting.

The guarantee

Under the algorithm's assumptions, the tokens are distributed **exactly** as if they had been sampled from the big model alone. Not approximately, and not "close enough in practice". The procedure is ordinary rejection sampling, so the result is identical by construction.

Consequences:

- a poor drafter mainly reduces the potential speed-up
- you still measure latency and run output regression tests
- the drafter can be anything: a small model, an *n*-gram table, the target's own extra heads

Constrained decoding

What it is

- Give the server a schema or grammar for the required output.
- At each step, mask tokens that would make the output invalid.
- Sample from the tokens that remain.

Why it is used

Applications often need parseable JSON, and retries add latency and cost.

What it guarantees

Valid syntax when generation completes successfully. The constraint removes invalid token sequences, but truncation and server failures still need handling.

What it does not guarantee

Anything about the content. The label inside the valid JSON can still be wrong, and the constraint can make it wronger.

Implementations such as **Outlines** and **XGrammar** make the allowed-token lookup efficient.

Sources: Willard & Louf, *Efficient Guided Generation for LLMs*, 2023 (Outlines); Dong et al., *XGrammar*, 2024.

Valid syntax can still reduce task accuracy

A grammar guarantees the *shape* of an output, not its content. The model may prefer to reason first, use another key order or decline to answer. Masking those continuations can force it onto a low-probability path and reduce answer quality.

	invalid output	task score
free generation + parser	4–15%	baseline
JSON-schema constrained	**0%**	**report it and see**

Report **both** invalid-output rate and task score. For reasoning models, let the model reason freely and constrain only the final answer.

Serving systems and cost

When one GPU is not enough

First identify the constraint that one GPU cannot meet.

The problem	The answer	What it costs
weights and KV do not fit, or decode is too slow	**split the model**	communication on every token
they fit, throughput is too low	**more replicas**	more hardware, little communication
MoE experts dominate memory	expert parallel	every GPU talks to every other

If the model fits and one copy is fast enough, run several independent replicas before splitting one request across GPUs. Replicas do not exchange activations and are easier to scale and recover.

Cost per million tokens, from first principles

$$\text{EUR per million tokens} = \frac{\text{EUR per GPU-hour}}{\text{tokens you got out of that hour}} \times 10^6$$

In words: you rent the card by the hour, so divide the rent by what the hour produced. Rent is EUR 2.50/h and the card is busy only 40% of the time (its **utilisation**), so everything divides by 0.4 at the end.

32 users at once, measured 2,500 tokens/s:

- $2500 \times 3600 = 9$ million tokens an hour
- EUR 2.50 for 9 million = EUR 0.28 per million
- busy only 40% of the time: **EUR 0.69 per million**

One user, measured 120 tokens/s: 0.43 million tokens an hour, the same arithmetic, **EUR 14.5 per million**.

The same GPU, the same model, 21× the price

Nothing changed except how many requests were in flight. Batch size is the dominant term in your unit economics.

Public prices in late 2026: roughly \$0.02–0.10 per million output tokens for hosted 4B-class open models, \$5–15 for frontier ones.

Source: GPU price: representative on-demand H100/GH200 cloud rate, late 2026. Throughput: the kind of figure Thursday's lab produces. API prices: public price lists, November 2026 — **these rot fast**.

Buying hardware for a decode-heavy workload

	bf16 TFLOP/s	bandwidth	memory
GH200 (96 GiB, HBM3)	$\approx$ 990	4.0 TB/s	96 GiB
GH200 (144 GB, HBM3e)	$\approx$ 990	4.9 TB/s	144 GB
A100 80 GB	$\approx$ 312	2.0 TB/s	80 GB
L40S	$\approx$ 362	0.86 TB/s	48 GB

Compute and bandwidth are the two halves of the roofline. The L40S has more advertised arithmetic than an A100 but less than half its memory bandwidth. A decode-heavy workload can therefore run faster on the A100.

Take away

For a decode-heavy workload, compare **bandwidth per euro** and memory capacity, not only peak arithmetic.

Research hook: deterministic inference

Three things to remember

1. **The two halves of a request hit different limits.** Prefill brings more arithmetic than the card can do at once, while decode finishes its arithmetic long before the weights have finished arriving. Find out which half you are paying for before you optimise, and remember that INT4 helps one of them and can hurt the other.

2. **The KV cache often sets the concurrency limit.** In our worked example, eight gigabytes hold the weights and most of the remaining card is available to the cache. Paging, prefix reuse and quantisation all reduce pressure on that budget.

3. **Measure latency and quality together.** Warm up the server, separate TTFT from TPOT, sweep concurrency, and report p50 and p95. A faster configuration is only useful if its output quality remains acceptable for the task.

Thursday: you serve a model, measure it, quantise it, and find out how close today's estimates were.

A server configuration you can reuse

If the same endpoint suits your project, it can become **infrastructure for later labs**:

- **Week 5, Dmitry's RAG lab**: your vLLM server is the *generator* that retrieval feeds.
- **Week 6, the agent lab**: the same endpoint, now driving a tool-calling loop with many short calls, so TTFT and prefix caching suddenly matter a lot.
- **Week 7**: your demo runs against it.

The file worth keeping

A one-command restart script in your repository. Model path, LoRA adapter, `max-model-len`, `max-num-seqs`, prefix caching and port, all of it in a file, so that in two weeks you type one line and the server you tuned today comes back exactly as it was.

The complete measurement workflow is in `lab3-example/README.md`.

Every year someone loses an hour of week 5 rebuilding a server command from memory. Do not be that person.

GPU etiquette, again: the endpoint holds its KV pool whether or not you are using it. Shut it down when you leave.

Reading

Serving systems

- Kwon et al. 2023, *PagedAttention* (vLLM)
- Yu et al. 2022, *Orca*, continuous batching
- Agrawal et al. 2024, *Sarathi-Serve*
- Zhong et al. 2024, *DistServe*
- Zheng et al. 2024, *SGLang* / RadixAttention
- Pope et al. 2023, *Efficiently Scaling Transformer Inference*

Kernels

- Dao et al. 2022, Dao 2023, *FlashAttention* 1 and 2
- Williams et al. 2009, the roofline model

Decoding

- Leviathan et al. 2023, speculative decoding
- Chen et al. 2023, speculative sampling
- Cai et al. 2024, *Medusa*; Li et al. 2024, *EAGLE*
- Willard & Louf 2023, *Outlines*; Dong et al. 2024, *XGrammar*

Quantisation

- Frantar et al. 2023, *GPTQ*; Lin et al. 2024, *AWQ*
- Hooper et al. 2024, *KVQuant*

Determinism

- Thinking Machines Lab 2025, *Defeating Nondeterminism in LLM Inference*
- SGLang deterministic-inference docs and issues